# EMNIST 데이터셋을 이용한 AE 기반 이상탐지 모델 최적화 및 분석

## (Optimization and analysis of AE-based anomaly detection model using EMNIST dataset)

숭실대학교 전자정보공학부 IT융합전공 20202982 위찬우, 20201599 이건아


## 요 약

산업 분야에서 이상탐지는 네트워크 보안의 비정상 트래픽 감지, 금융 시스템의 비정상 거래 탐지 등 다양한 영역에서 중요한 역할을 한다. 이러한 이상 탐지에서 오토인코더(Autoencoder, AE)는 정상 데이터만을 학습하고 비정상 데이터 입력 시 큰 복원 오차(Reconstruction Error)를 통해 이상 여부를 판별하는 데 효과적으로 활용된다. 본 연구에서는 EMNIST 데이터셋을 활용하여 AE 기반의 이상탐지 모델을 개발하고 최적화하였다. 모델은 숫자 데이터를 학습하여 알파벳 문자를 이상치로 인식하도록 설계되었다. 또한, 배치 크기, 학습 횟수, 학습률, 레이어 크기 등의 다양한 하이퍼파라미터를 조정하여 성능을 극대화하였고, 손실 함수(MSE, Huber), 활성화 함수(ReLU, Leaky ReLU), 최적화 알고리즘(Adam, NAdam) 등의 조합을 비교 분석하였다. 최적의 구성으로 선택된 5층 AE 모델은 크기가 32인 잠재 공간과 ReLU 활성화 함수를 적용하여 높은 재현율과 F1-스코어를 기록하였다. 본 연구는 AE 기반 이상탐지 모델의 성능 향상에 있어 하이퍼파라미터 최적화의 중요성을 강조하며, 이상탐지 모델의 활용 가능성을 제시한다.

## ABSTRACT

Anomaly detection plays a crucial role in various industrial fields, such as detecting abnormal traffic in network security and identifying fraudulent patterns in financial systems. In such anomaly detection tasks, autoencoders (AEs) are effectively used to learn only normal data, allowing the model to detect anomalies by identifying significant reconstruction errors when abnormal data is input. This study develops and optimizes an AE-based anomaly detection model using the EMNIST dataset. The model is designed to learn numeric data and recognize alphabetic characters as anomalies. To maximize performance, various hyperparameters, including batch size, number of epochs, learning rate, and layer dimensions, were systematically adjusted, and combinations of loss functions (MSE, Huber), activation functions (ReLU, Leaky ReLU), and optimization algorithms (Adam, NAdam) were compared and analyzed. The optimal configuration, a 5-layer AE model with a latent space size of 32 and ReLU activation, achieved high recall and F1-scores. This study highlights the importance of hyperparameter optimization in enhancing the performance of AE-based anomaly detection models and demonstrates the potential for their practical applications in anomaly detection.


키워드 : 이상탐지, 과적합 방지, 복원 오차, 하이퍼파라미터 최적화
Key Words : Anomaly Detection, Overfitting Prevention, Reconstruction Error, Hyperparameter Optimization

# I. 서론

산업 분야에서 이상탐지는 네트워크 보안에서 비정상 트래픽을 감지하거나, 금융 시스템에서 비정상 거래 패턴을 파악하는 등 중요한 역할을 한다. 이러한 이상 탐지에서 오토인코더(AutoEncoder)는 주로 정상 데이터를 학습하여 이상치가 발생했을 때 큰 복원 오차(Reconstruction Error)를 통해 이상 여부를 감지하는 데 사용된다.

오토인코더(AutoEncoder)는 주어진 데이터를 인코딩(Encoding)하여 저차원 공간에 압축하고, 이를 디코딩(Decoding)하여 원본 데이터에 가깝게 복원하는 비지도 학습 기반 신경망 모델이다[1]. 입력 데이터와 출력 데이터의 차이를 최소화하는 과정을 통해 중요한 특징을 학습하며, 특히 정상 데이터의 패턴을 학습한 모델은 이상 데이터를 입력받을 경우 큰 복원 오차(Reconstruction Error)를 보이는 특성을 갖는다. 이 특성은 오토인코더가 이상탐지 모델로서 활용되는 핵심 원리로, 본 프로젝트에서는 EMNIST 데이터셋의 숫자 데이터를 학습하여 정상 패턴을 학습하고, 문자를 이상으로 간주하여 이를 탐지하는 데 오토인코더를 적용하고자 한다.

# II. 본론

본 연구는 EMNIST 데이터를 활용하여 이상탐지 성능에 최적화된 오토인코더 모델을 설계하고, 하이퍼파라미터를 조정하여 이상 데이터를 효과적 으로 탐지하는 것을 목표로 한다. 모델의 주요 구성 요소는 각 층의 활성화 함수, 손실 함수, 최적화 기법, 잠재 벡터 크기 등으로, 각각이 모델의 학습 과정과 최종 이상 탐지 성능에 미치는 영향을 분석하였다. 이러한 구성 요소들은 모델의 성능을 극대화하는 중요한 역할을 수행하며, 이상치 데이터에 대해 높은 민감도를 유지하는 데 기여한다. 또한, 하이퍼파라미터는 모델 학습의 효율성과 성능에 직결되므로, 이들을 유기적으로 고려하며 설정해야 한다. 본론에서는 batch size, epochs, learning rate, 레이어 수 등 주요 하이퍼파라미터를 조정하여 최적의 이상탐지 성능을 도출하고자 한다.

## 1. 모델의 구성요소

### 1.1 손실 함수 (Loss Function)

손실 함수는 예측 결과와 실제 값 간의 차이를 측정해 모델의 학습 방향을 결정짓는 역할을 한다. 본 연구에서는 손실 함수로 MSE와 Huber Loss를 선택하여, 두 함수가 각각 모델 성능에 미치는 영향을 비교하고자 한다.

#### 1.1-1 MSE(Mean Sqaured Error)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \quad (1)$$

평균 제곱 오차(MSE)는 예측 값과 실제 값 사이의 차이를 제곱하여 평균을 구하는 방식이다. 오차가 커질수록 손실이 급격하게 증가하여 큰 오차에 패널티를 부과하는 특징이 있다.

#### 1.1-2 Huber Loss

$$L_\delta(e) = \begin{cases} \frac{1}{2}a^2 & for\ |a| \le \delta, \\ \delta\,(|a| - \frac{1}{2}\delta), & otherwise. \end{cases} \quad (2)$$

Huber Loss는 MSE와 절대 오차(Mean Absolute Error)의 특성을 결합하여 이상치에 더 강한 성능을 보인다. delta 매개변수를 통해 오차가 작을 때는 MSE처럼, 클 때는 절대 오차처럼 동작하여 과도한 패널티를 부과하지 않도록 한다. 이로 인해 데이터에 이상치가 포함된 경우에도 안정적인 학습이 가능하다.




그림 1. MSE와 Huber Loss의 비교
Fig. 1. Comparison of MSE and Huber Loss

본 연구의 최종 모델에서는 delta값이 1.0인 huberloss가 선택되었다.

### 1.2 활성화 함수(Activation Function)

활성화 함수는 각 뉴런의 출력을 결정하며, 비선형성을 학습하게 한다. 본 연구에서는 ReLU와 Leaky ReLU를 비교하여 모델의 성능을 극대화할 수 있는 활성화 함수를 선택하였다.

#### 1.2-1 ReLU (Rectified Linear Unit)

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \le 0) \end{cases} \quad (3)$$

ReLU는 입력이 양수일 때는 그대로 출력하고, 음수일 때는 0으로 출력하는 함수다. 이 함수는 연산이 간단하고 학습 속도가 빠르다는 장점이 있어, 특히 깊은 신경망에서 일반적으로 사용된다.

#### 1.2-2 Leaky ReLU

$$f(x) = \begin{cases} x & if\ x > 0, \\ 0.01x & otherwise. \end{cases} \quad (4)$$

Leaky ReLU는 ReLU의 변형으로, 음수 입력에 대해 작은 양수 값을 반환하여 뉴런이 학습에서 완전히 배제되지 않도록 한다. 하지만 입력값이 음수가 없는 MNIST 데이터 특성상 ReLU와 크게 차이가 나지 않았고, 심지어 최종 모델에서 약간의 성능 저하를 보여 선택되지 않았다.

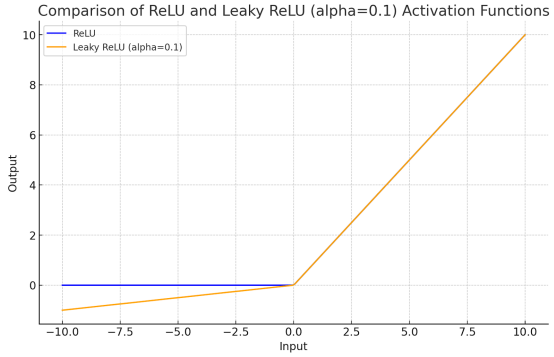



그림 2. ReLU와 Leaky ReLU의 비교
Fig. 2. Comparison of ReLU and Leaky ReLU

### 1.3 최적화 기법(Optimizer)

최적화 기법은 학습 과정에서 모델의 가중치를 조정하는 방식으로, 최적화 기법에 따라 학습의 수렴 속도와 최종 성능이 달라질 수 있다. 본 연구에서는 Adam과 NAdam을 사용하여 최적화 기법이 성능에 미치는 영향을 분석하였다.

#### 1.3-1 Adam(Adaptive Moment Estimation)

$$m_t = \beta_1 m_{\{t-1\}} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 g_{\{t-1\}} + (1 - \beta_2) g_t^2$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v}_t} + \epsilon} \widehat{m}_t \qquad (5)$$

Adam은 학습률을 각 변수마다 다르게 조정하여 빠르고 효율적으로 최적화를 수행한다. 과거와 현재의 기울기를 조합하여 가중치를 업데이트하는 방식을 사용한다.

#### 1.3-2 NAdam (Nesterov Adam)

$$g_t = \nabla_{\theta_t} J(\theta_t)$$

$$m_t = \gamma m_{t-1} + \eta g_t$$

$$\theta_{t+1} = \theta_t - m_t$$

$$\therefore \theta_{t+1} = \theta_t - (\gamma m_{t-1} + \eta g_t) \qquad (6)$$

NAdam은 Adam의 변형으로, Nesterov Momentum을 적용하여 최적화 성능을 향상시킨다. 이 기법은 기울기 방향을 미리 반영해 더 빠르고 예측 가능한 경로로 학습이 이루어지도록 유도하며, 특히 빠른 수렴과 안정성을 원하는 경우에 효과적이다. 따라서 본 연구에서는 최종적으로 NAdam을 사용하였다.

### 1.4 잠재 벡터 (Latent Vector)

잠재 벡터는 입력 데이터의 주요 정보를 압축하여 담고 있는 벡터로, 모델의 압축 성능과 복원 성능의 균형을 결정짓는 중요한 요소다. 본 프로젝트에서는 잠재 벡터 크기를 주어진 32로 고정하여 실험을 진행하였다.

## 2. 하이퍼파라미터

### 2.1 배치 크기 (Batch Size)

배치 크기는 학습 시 한 번에 처리하는 데이터의 묶음 크기를 뜻하며, 이를 통해 학습을 더 빠르고 효율적으로 진행할 수 있다. 큰 배치 크기는 학습 속도가 빠르고 안정적인 학습이 가능하지만, 일부 데이터 패턴을 놓칠 수 있으며 GPU 자원을 많이 소모한다. 반면, 작은 배치 크기는 메모리 사용이 적고 세부 패턴을 잘 학습할 수 있지만, 학습이 불안정해질 수 있다.

### 2.2 학습 횟수(Epochs)

epoch는 전체 데이터셋을 학습하는 횟수를 의미하며, 학습의 깊이와 과적합 방지에 중요한 역할을 한다. 많은 epoch는 복잡한 패턴을 깊이 학습할 수 있지만, 과적합 위험이 커질 수 있다. 반면 적은 epoch는 과적합을 줄이지만 학습이 충분하지 않아 성능이 저하될 수 있어, 데이터에 맞는 적정 epoch 설정이 중요하다.

### 2.3 학습률(Learning Rate)

학습률은 모델의 가중치가 갱신되는 비율로, 학습의 속도와 안정성을 결정짓는 중요한 요소이다. 높은 학습률은 모델 최적화를 빠르게 하지만 학습이 불안정할 수 있다. 낮은 학습률은 안정적이나 학습 속도가 느려지므로, 모델의 학습 속도와 안정성을 고려해 적절히 조정해야 한다.

### 2.4 Hidden Layer 수

모델의 hidden layer 수는 모델의 표현력과 계산 자원의 소모에 영향을 미치며, 오토인코더에서는 인코더와 디코더의 레이어 구성이 데이터 압축과 복원 성능에 중요한 역할을 한다.
Layer 수가 많으면 데이터의 세밀한 구조를 잘 학습할 수 있지만, 과적합 위험과 높은 계산 자원 소모가 있다. Layer 수가 적으면 연산 비용이 줄고 학습 속도가 빨라지지만 복잡한 패턴 학습이 부족해 정확도가 낮아질 수 있다.

### 2.5 Layer의 차원 수

Layer의 차원 수는 모델의 용량과 정보 표현력에 영향을 주며, 특히 오토인코더 모델에서 차원 수는

입력 데이터를 압축하고 복원하는 능력에 중요한 역할을 한다.

큰 차원 수는 데이터의 세부적인 패턴을 유지하고 복잡한 구조를 학습하는 데 유리하지만, 과도하게 크면 중요한 정보의 압축 효과가 떨어지고 계산 비용이 증가할 수 있다. 반면, 작은 차원 수는 연산이 효율적이지만 정보 손실로 인해 중요한 패턴이 충분히 반영되지 않아 모델 성능이 저하될 수 있다. 데이터의 복잡성과 모델 목표에 맞춰 적절한 차원 수를 설정하는 것이 중요하다.

## III. 실험

본 실험은 1차 성능 평가에서 validation set의 성능을 극대화하려는 과정에서 얻은 인사이트를 바탕으로 test set에서 이상치 탐지의 퍼포먼스를 최대한으로 끌어 올리는 방향으로 진행되었다. 전반적인 밸런스를 고려하였고, 그 중에서도 precision과 recall의 절대적인 수치와 균형을 둘 다 고려하는, 따라서 모델의 성능을 대표한다고 볼 수 있는 지표가 f1-score라고 생각되어 f1-score를 주 성능지표로 뽑았다[2]. 그리고 실제 산업이나 실무 에서는 validation set이 모델의 잠재 성능을 탐색 하는 단계로 활용되는 반면, test set에서의 성능이 모델의 일반화 성능과 실제 사용 가능성을 평가하는 데 중요한 기준이 되기 때문에 test set에 조금 더 중점을 두고 진행했다.

모델의 구성 요소인 loss function, activation function, optimizer 등은 처음부터 EMNIST를 처리하는 Autoencoder에 적합하다고 알려진 2~3개의 옵션들로 선택하여 비교하였기 때문에 이들 구성 요소 간의 성능 차이는 상대적으로 미미한 수준이었다. 때문에 epoch, batch size, learning rate, Layer의 수와 차원 수 등의 주요 하이퍼 파라미터를 조정하여 모델의 기본 성능을 먼저 확보 하였다. 이후, 모델 구성 요소를 통해 미세한 성능 조정을 시도함으로써 최적화를 도모하였다.

### 1. 데이터 셋

본 연구에서는 EMNIST 데이터셋을 사용하여 이상 탐지 모델을 설계하였다. 구체적으로, 학습(train) 데이터셋은 숫자 이미지로만 구성하여 오토인코더가 정상 패턴을 학습하도록 하였으며, 검증(validation) 및 테스트(test) 데이터셋은 숫자와 영문자 이미지를 모두 포함하여 혼합된 형태로 구성하였다[3]. 이때 숫자는 정상 데이터로, 문자는 이상 데이터로 간주하여 오토인코더가 영문자를 비정상 패턴으로 감지하도록 유도하였다. 이러한 구성은 실제 산업에서 정상적인 패턴을 학습한 모델이 새로운 데이터에서 비정상적인 입력을 감지하는 것과 유사한 설정으로, 오토인코더 기반의 이상 탐지 성능을 효과적으로 평가할 수 있도록 설계되었다.

### 2. 하이퍼파라미터

#### 2.1 학습 횟수(Epoch)

epoch 크기를 늘릴수록 validation set에 대한 성능은 향상되었으나, 이에 비례해 과적합이 심해져 test set에서의 성능이 크게 저하되었다. 반면, 초기값으로 10을 설정했을 때는 학습이 충분히 이루어지지 않아 과소적합이 발생했고, 이로 인해 validation set과 test set 모두에서 성능이 저조했다. 여러 번의 시행착오 끝에 epoch을 100으로 설정하는 것이 가장 적절하다고 결론을 내렸다.

표 1. epoch에 따른 F1-score
Table 1. F1-score by epoch

| num_epochs | *valid_F1-Score* | *test_F1-Score* |
|---|---|---|
| 50 | 0.8344 | 0.797 |
| 100 | 0.8757 | 0.8127 |
| 150 | 0.8981 | 0.8027 |

#### 2.2 배치 크기(Batch Size)

batch size는 2의 제곱수가 메모리와 호환성이 좋아 32, 64, 128 정도가 적절하기 때문에 2의 제곱수로 실행해 보았다. 최종 모델에서의 실험 결과 대부분의 경우에서 batch size 32은 64에 비해 recall을 제외한 대부분의 지표에서 비교적 성능이 저하되는 경향을 보였다. 또한, 128와 64는 근소한 성능 차이를 보였으나, 최종적으로 layer 구성이 확정된 이후에는 batch size 64가 더 우수한 성능을 보인다고 결론을 내렸다.

표 2. batch_size에 따른 F1-score
Table 2. F1-score by batch_size

| batch_size | *valid_F1-Score* | *test_F1-Score* |
|---|---|---|
| 32 | 0.8872 | 0.8053 |
| 64 | 0.8757 | 0.8127 |
| 128 | 0.8694 | 0.8109 |

#### 2.3 학습률(Learning Rate)

learning rate는 일반적으로 설정되는 0.01부터 시작해 보았으나, 성능이 현저히 낮고 학습이 불안정하여 학습률을 낮추는 방향으로 조정하였다. 이에 따라 0.001부터 0.0001까지 여러 값을 실험한 결과, 5e-04에서 가장 안정적인 성능을 보인다는 결론을 내렸다.

표 3. learning_rate에 따른 F1-score
Table 3. F1-score by learning_rate

| learning_rate | *valid_F1-Score* | *test_F1-Score* |
|---|---|---|
| 1.00E-04 | 0.8158 | 0.7819 |
| 5.00E-04 | 0.8757 | 0.8127 |
| 1.00E-03 | 0.8883 | 0.8058 |

#### 2.4 Hidden Layer 수 & Layer 차원 수

Hidden Layer의 수와 각 Layer의 차원 수는 본 연구에서 가장 성능에 큰 영향을 미치는 하이퍼파라미터로, 조정에 따라 성능 차이가 크게 나타났다.

hidden layer가 3개일 때는 데이터의 복잡한 패턴을 표현하지 못해 과소적합이 나타났고, 4개로 증가하면서 validation 성능이 개선되었으나 test 성능에서 한계가 있었다. hidden layer가 5개일 때는 과적합이 상대적으로 줄어들고, 안정적인 test 성능을 보였다. 그러나 6개로 증가할 경우 모델이 과도하게 복잡해져 과적합이 발생했다. 따라서 validation과 test set에서 균형 잡힌 성능을 위해 최종적으로 hidden layer 5개가 가장 효과적이었다.

표 4.hidden layer 수에 따른 F1-score
Table 4. F1-score by number of hidden layers

| fisrt_hl | second_hl | third_hl | forth_hl | fifth_hl | valid_F1-Score | test_F1-Score |
|---|---|---|---|---|---|---|
| 600 | 450 | 250 | | | 0.896 | 0.7684 |
| 600 | 450 | 250 | 150 | | 0.8991 | 0.8 |
| 600 | 450 | 250 | 150 | 80 | 0.8757 | 0.8127 |

표 5.layer 차원수에 따른 F1-score
Table 5. F1-score by dimension oflayers

| fisrt_hl | second_hl | third_hl | forth_hl | fifth_hl | valid_F1-Score | test_F1-Score |
|---|---|---|---|---|---|---|
| 580 | 400 | 240 | 120 | 60 | 0.8765 | 0.8023 |
| 600 | 450 | 250 | 150 | 80 | 0.8757 | 0.8127 |
| 620 | 470 | 280 | 150 | 70 | 0.8771 | 0.8061 |

## IV. 실험 결과

### 1. 최종 모델 분석

표 5. 모든 하이퍼 파라미터에 따른 performance
Table 5. Performance by Hyperparameter tuning

| batch_size | num_epochs | learning_rate | fisrt_hl | second_hl | third_hl | forth_hl | fifth_hl |
|---|---|---|---|---|---|---|---|
| 64 | 100 | 5.00E-04 | 600 | 450 | 250 | 150 | 80 |

| latent_size=32 | act_fn | loss_fn | opt |
|---|---|---|---|
| 32 | ReLU | HuberLoss(delta = 1.0) | NAdam |

| Loss | valid_threshold | valid_Accuracy | valid_Precision | valid_Recall | valid_F1-Score |
|---|---|---|---|---|---|
| 0.0058 | 0.0716 | 0.8825 | 0.9292 | 0.8281 | 0.8757 |

| test_Accuracy | test_Precision | test_Recall | test_F1-Score |
|---|---|---|---|
| 0.8097 | 0.8001 | 0.8257 | 0.8127 |

최종적으로 선정된 오토인코더 모델은 Hidden Layer 5개로 구성되었으며, 각 Layer의 차원 수는 데이터의 복잡한 패턴을 효과적으로 학습할 수 있도록 최적화되었다. 학습률(Learning Rate)은 5e-04로 설정하여 학습의 안정성을 높혔고, 배치 크기(Batch Size)는 64로 설정하여 성능과 학습 안정성 간의 균형을 맞추었다. 또한, 손실 함수로 Huber Loss를 사용하였으며, 활성화 함수로는 ReLU를 채택하여 모델의 학습 속도와 효율성을 극대화하였다. 이 모델 구성은 EMNIST 데이터셋에서 정상 패턴을 안정적으로 학습하고, 이상 데이터에 대해서는 높은 복원 오차를 발생시켜 이상탐지 성능을 향상시키는 데 적합하였다.

최종 모델의 성능 평가 결과, validation set과 test set에서 각각 높은 F1-score를 달성하였으며, 정확도(Accuracy), 정밀도(Precision), 재현율(Recall)에서도 개선된 성능을 확인하였다. 특히, test set에서는 오탐과 미탐을 최소화하며 이상 데이터를 효과적으로 감지하는 성능을 보여, 모델이 실제 환경에서도 신뢰성 있게 이상 탐지를 수행할 수 있음을 확인하였다. 또한, 다양한 성능 지표 분석을 통해 F1-score가 향상됨에 따라, 이상탐지의 민감도와 정확도를 모두 고려한 균형 잡힌 성능을 얻을 수 있었다. 이를 통해 최적화된 하이퍼파라미터 와 모델 구성은 EMNIST 데이터셋 기반 이상탐지 작업에 있어 최적의 성능을 발휘함을 확인할 수 있었다.

### 2. Epoch에 따른 Learning Curve 분석

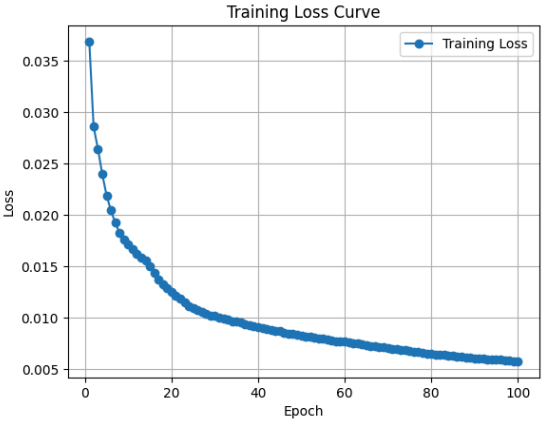



그림 3. 최종 모델의 학습곡선
Fig. 3. Learning curve of model

주어진 Training loss curve를 보면, epoch가 진행됨에 따라 초반엔 급격하게, 중반부터는 서서히 Training Loss가 감소하고 있어 모델이 점진적으로 데이터를 더 잘 예측하고 있음을 알 수 있다. epoch가 진행될수록 감소율이 둔화되는 양상 이 나타나며, 이는 모델이 수렴 단계에 도달하고 있음을 시사한다. 그래프에서 loss의 급격한 변동이 없어 학습이 안정적으로 진행되고 있음을 확인 할 수 있으며, 이는 현재 설정된 하이퍼파라미터가 적절하게 설정된 결과일 가능성이 있다. loss가 완전히 수렴하지 않았으므로 본 연구에서 제한된 다른 모델 사용 등을 통해 추가적인 성능 향상은 충분히 가능할 것으로 예상된다.

## V. 결론

본 프로젝트에서 오토인코더의 다양한 파라미터를 통해 모델 구조를 변경해 가며 최적의 성능을 보여주는 이상탐지 모델을 만드는 것을 목표로 하였다. 주어진 환경에서 최선의 모델을 만들었다고 생각하지만, 실제 산업계에서 쓰기엔 개선할 점이 많은 모델이라 판단된다. 따라서 실제 산업에 적용되는 오토인코더 모델, 예를 들면 Convolution al, Variational, Den oising 등 특정 도메인의 요구사항과 데이터 특 성에 맞춰 더 다양한 구조와 최적화 기법을 사용하여 성능을 극대화할 수 있을 것으로 보인다.

## Reference

[1] Sangchul Yoo, “Performance Comparison of Autoencoder Anomaly Detection Algorithms.” Master’s thesis, Ajou University, 2023, Gyeonggi-do, South Korea. (https://www.riss.kr/search/detail/DetailView.do?p_mat_type=be54d9b8bc7cdb09&control_no=3ccaaba61dd227ecffe0bdc3ef48d419&keyword=%EC%98%A4%ED%86%A0%EC%9D%B8%EC%BD%94%EB%8D%94)
[2] Miru Kim, Hyoseon Kye, Minhae Kwon,Network Anomaly Detection System Using Hidden Layer Information of Autoencoder, *The Journal of Korean Institute of Communications and Information Sciences '22-09 Vol.47 No.09*(https://doi.org/10.7840/kics.2022.47.9.1310)
[3] S. K. Dana, “Incremental Learning of Handwritten Characters in EMNIST dataset,” *2023 International Conference for Advancement in Technology* (ICONAT) (https://ieeexplore.ieee.org/document/10080724)

https://comdon-ai.tistory.com/49 : 함수 식 참고
https://aws.amazon.com/ko/what-is/hyperparameter-tuning/ : 하이퍼파라미터 튜닝이란?

## colab link

https://colab.research.google.com/drive/1_Dv3HvpIcIHxbaYOyMuxGSvronZABZN6#scrollTo=RwKtyvrPQ5M5